

Vehicle Image Quality Classification

1. Background

Automated vehicle inspection systems used in domains such as insurance, rentals, fleet management, and logistics depend heavily on clear, high-quality images to identify scratches, dents, and other forms of damage accurately. In real-world usage, however, captured images are often affected by poor environmental or capture conditions.

Two of the most common issues are:

- **Blur** - caused by motion, focus problems, or lens smudging
- **Low Light** - caused by underexposure or noisy imaging conditions that hide surface-level details

When such images are passed into downstream damage-detection systems, the reliability of the inspection drops significantly. To address this, we need an image-quality screening step that can determine whether an image is suitable for further analysis.

2. Objective

Design and implement a computer vision solution that classifies vehicle images based on their suitability for inspection.

The system should identify the following image-quality failure modes:

- **Blur:** Includes motion blur, out-of-focus capture, and blur caused by lens contamination or smudging
- **Low Light:** Includes underexposed images or images captured in poor lighting conditions where important vehicle details are no longer clearly visible

The goal is to ensure that poor-quality images can be flagged before they are passed to downstream inspection or damage-analysis models.

3. Technical Requirements

The solution should include the following:

Inference Script

A Python-based inference script that accepts a directory of images as input and generates an output file in **JSON** or **CSV** format containing:

- image filename
- predicted class or labels
- confidence score(s)
- final **Accept / Reject** status

Thresholding

The system should support a configurable confidence threshold (for example, $\tau = 0.7$) that determines whether an image is considered acceptable for downstream inspection.

4. Evaluation and Metrics

The model will be evaluated not only on raw predictive performance, but also on how reliably it can function in a production-style setting.

Please provide a report that includes the following:

Quantitative Metrics

- **Per-class Precision and Recall**

Recall is especially important in this use case, since low-quality images should not be allowed to pass through undetected.

- **F1-Score**

This is important to assess model performance in the presence of class imbalance.

- **Inference Latency**

Average time taken to process a single image (for example, at input sizes such as **224 × 224** or **512 × 512**).

- **Confusion Matrix**

To help analyze model behavior, especially in cases where one failure mode may be confused with another.

Qualitative Analysis

- **Failure Analysis**

Select 3–5 examples where the model produced incorrect predictions and briefly explain why the model may have failed in those cases.

- **Edge Cases**

Discuss how the system handles borderline or ambiguous scenarios, such as partial blur, uneven lighting, or mixed-quality images.

5. Deliverables

Please submit the following:

- **Codebase**

Clean, well-structured, and documented Python code, shared as a GitHub repository or ZIP file

- **Model Artifacts**

Saved model weights or exported model files

- **Performance Report**

A PDF or Markdown report containing the evaluation metrics and analysis described

above

- **Usage Instructions**

A README.md explaining environment setup, dependency installation, and how to run inference

6. Constraints

- **Execution:** The solution should be practical to run on a standard mid-range GPU, and efficient CPU execution is a plus